Producer-Consumer

son: 5 · Concurrency · Category: Concurrency · Level: Intermediate · Build with W

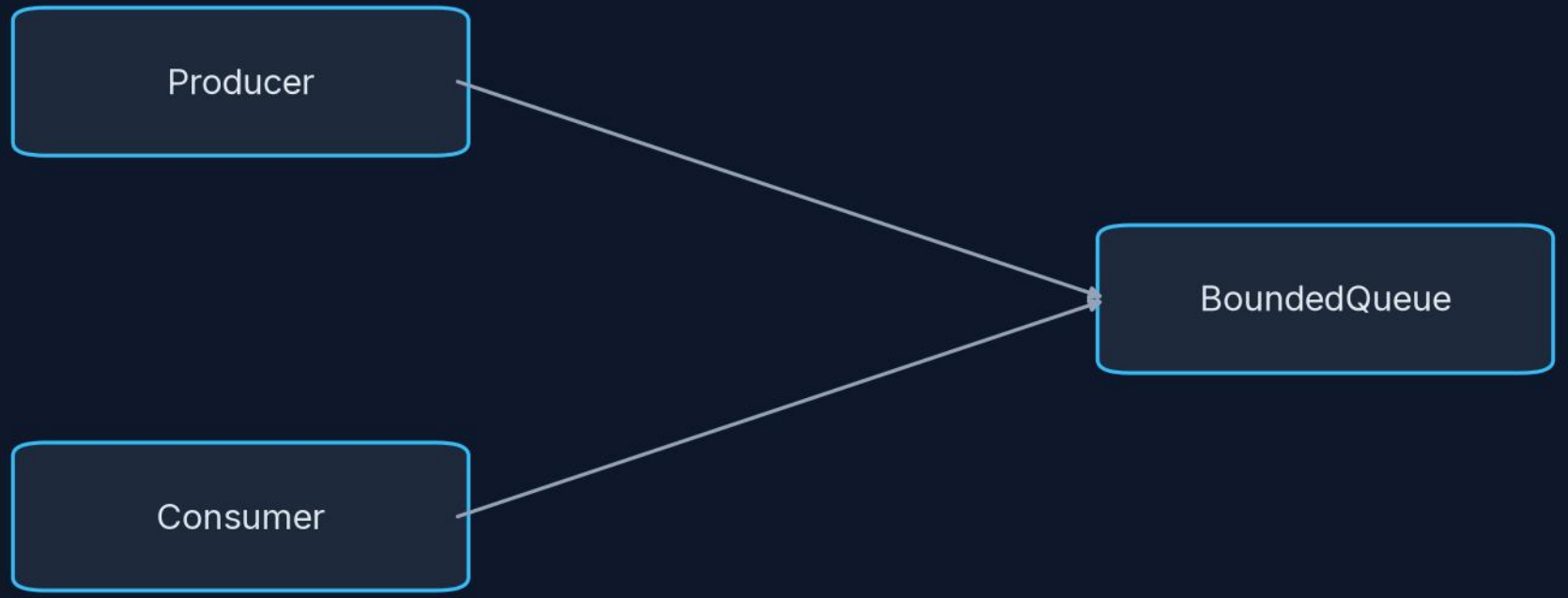
Today's Hook

- Today: Producer-Consumer as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Producer-Consumer structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Producer-Consumer
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/concurrency/producerconsumer/BoundedQueue.java
package com.buildwithwaqar.patterns.concurrency.producerconsumer;

import java.util.ArrayDeque;
import java.util.Queue;

public final class BoundedQueue<T> {
    private final Queue<T> q = new ArrayDeque<>();
    private final int capacity;

    public BoundedQueue(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (q.size() == capacity) wait();
        q.add(item);
        notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while (q.isEmpty()) wait();
        var item = q.remove();
        notifyAll();
        return item;
    }
```

Implementation (2)

```
    public synchronized int size() { return q.size(); }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.concurrency.producerconsumer;
import java.util.concurrent.*;
public final class Demo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Producer-Consumer walkthrough ===");
    var q = new BoundedQueue<Integer>(2);
    try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
      pool.submit(() -> { for (int i = 1; i <= 5; i++) { q.put(i); System.out.println("put " + i); } return null; });
      var sum = pool.submit(() -> { int s=0; for (int i=0;i<5;i++){ int v=q.take(); System.out.println("take "+v); s+=v;} return s; });
      System.out.println("sum=" + sum.get());
    }
  }
}
```

Walkthrough Output

```
=== Producer-Consumer walkthrough ===
take 1
put 1
put 2
put 3
take 2
take 3
put 4
put 5
take 4
take 5
sum=15
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Producer-Consumer
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Producer-Consumer → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.